

Project 2: Image Stitching

張晴昀 (B10902064)

黃允謙 (B10902067)

May 2, 2025

1 Description

This project aims to stitch a set of images into a seamless panorama. The process involves several key image processing steps, including feature detection, description, and matching. Once corresponding features are matched across images, further operations such as warping, blending, alignment and cropping are performed to generate the final panoramic output. Image stitching enhances both the effective resolution and the field of view, making it a powerful technique for producing wide-angle visual representations.

2 Processing Flow

1. Feature Detection
 - Moravec Corner Detector
 - Harris Corner Detector (Bonus)
2. Feature Description
3. Feature Matching
4. Image matching
5. Blending
6. End-to-end Alignment (Bonus)
7. Rectangling (Bonus)

3 Feature Detection

3.1 Moravec Corner Detector

We first implement the simple Moravec corner detector. For each pixel, the algorithm calculates the intensity difference between its patch and a shifted version. Then, it uses the minimum difference among four shifted directions as the pixel's energy. A corner exhibits drastic intensity changes in all directions, and thus results in higher energy.

$$E(\mathbf{u}) = \sum_X w(\mathbf{x}) [I(\mathbf{x} + \mathbf{u}) - I(\mathbf{x})]^2$$

3.2 Harris Corner Detector

However, Moravec has several problems. First, it uses a binary window function for the patch, which makes it more susceptible to noise. Second, it only considers shifts in eight directions, potentially missing corners at other angles.

The Harris corner detector introduces an alternative method for computing the energy to address these issues. It uses a Gaussian function to produce a smoother window for the patch. Then, it applies Taylor expansion to approximate the intensity difference for a given shift direction, which can be expressed as:

$$E(\mathbf{u}) \approx \mathbf{u}^\top \mathbf{M} \mathbf{u}, \quad \mathbf{M} = \sum_X w(\mathbf{x}) \begin{bmatrix} I_x^2 & I_x I_y \\ I_y^2 & I_y^2 \end{bmatrix}$$

Since E is a quadratic function of the shift vector $\mathbf{u}$, we can analyze its shape by examining the properties of $\mathbf{M}$. As a result, Harris considered the eigenvalues of $\mathbf{M}$. When both eigenvalues are large, it indicates that E will be large in all directions $\mathbf{u}$, suggesting the presence of a corner. Harris proposed the following energy function:

$$E = \det \mathbf{M} - k(\text{tr} \mathbf{M})^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

3.3 Comparison

In Figure 1, we demonstrate the results of the Moravec and Harris algorithms. In all the images, redder colors indicate higher values, while bluer colors indicate lower values.

Both algorithms performs well in identifying the corners or possible feature locations. In the Harris algorithm column, we observe that the horizontal and vertical lines appear in blue, which aligns with our expectations. However, we did not anticipate that a diagonal line would appear in red and be classified as a corner.

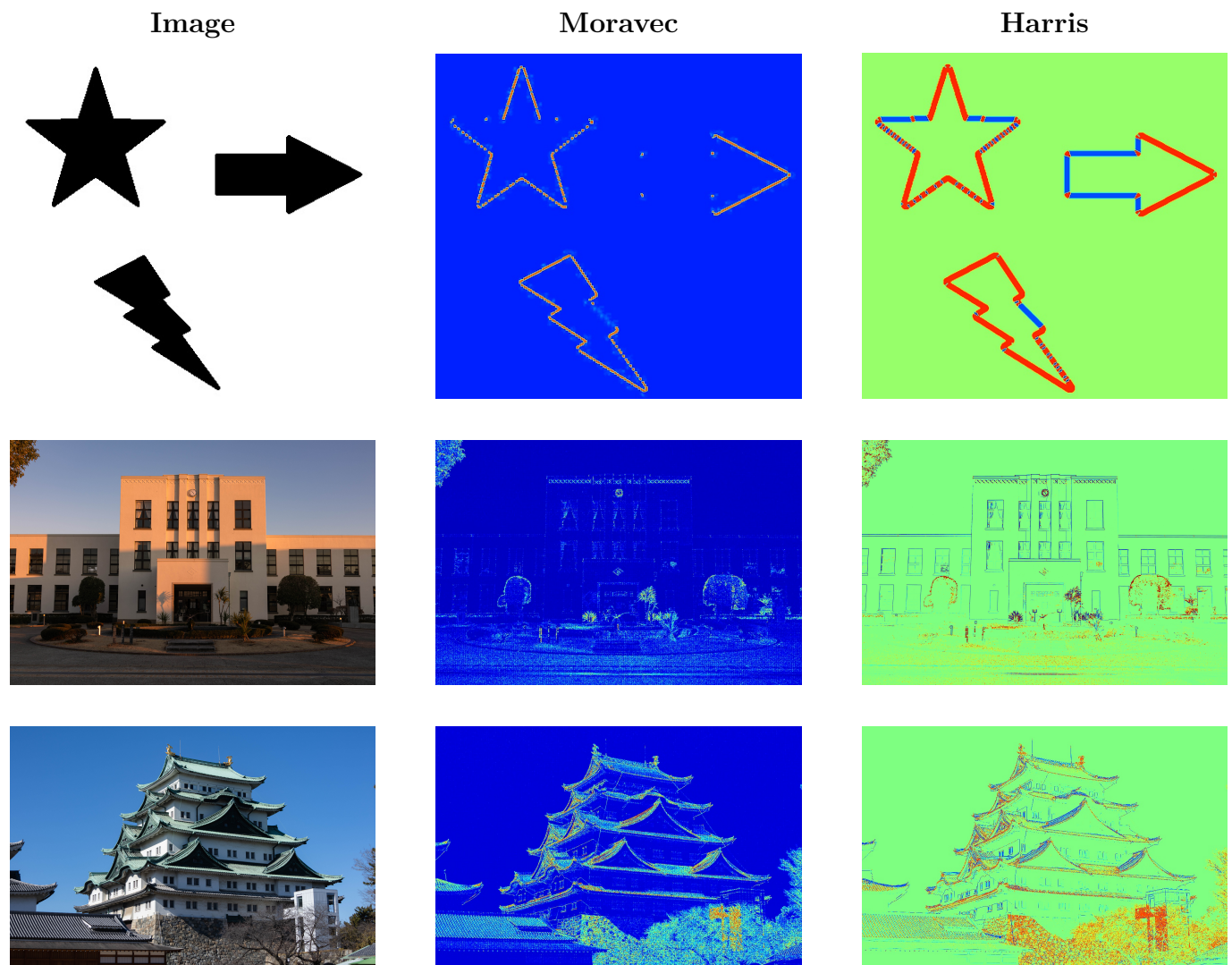


Figure 1: Comparison of Moravec and Harris Algorithms

3.4 Threshold

3.5 Maximal Suppress

3.6 Interactive Gadget

4 Feature Description

5 Feature Matching

6 Image Matching

To perform image matching, we estimate a 2D translation transformation that aligns two images based on a set of matched feature points using the RANSAC algorithm. The translation model assumes that the transformation between corresponding points in the two images can be described by a vector (d_x, d_y) , where each point (x, y) in the first image maps to $(x + d_x, y + d_y)$ in the second image. RANSAC is used to estimate this translation by iteratively selecting minimal subsets of point correspondences (in this case, one pair is sufficient for translation), computing the translation vector from the subset, and identifying inliers that agree with the estimated model within a predefined threshold. The model with the highest number of inliers is selected as the final translation estimate.

6.1 Threshold

7 Blending

To seamlessly combine the aligned images, we apply linear blending in the overlapping region. After estimating the translation, the overlapping area between two images is computed. In this region, pixel values are blended using a weighted sum, where the weights vary linearly across the overlap to ensure a smooth transition. Specifically, for each pixel p in the region, the final value is computed as

$$I(p) = \alpha(p_1) \cdot I_1(p_1) + (1 - \alpha(p_1)) \cdot I_2(p_2),$$

where I_1 and I_2 are the intensity values from the two images, p_1 and p_2 are the corresponding pixels in the two images, and $\alpha(p) \in [0, 1]$ is a spatially varying weight based on the pixel's relative position within the overlapping area. In our implementation, we assume the first image is on the left and the second image is on the right. The weight $\alpha(p_1)$ is determined based on the pixel's horizontal distance from the right edge of the first image: pixels at the edge are assigned a weight of 0, pixels at the start of the overlapping area are assigned 1, and those in between are weighted linearly between $[0, 1]$.

8 End-to-end Alignment

To reduce visual discontinuities caused by vertical shift, we implement an end-to-end alignment strategy that distributes the vertical shift across image columns. After stitching all images to form a panorama, we apply a column-wise vertical adjustment. For example, if the resulting panorama is tilted toward the lower right, we shift the first column downward by a specified amount, while the last column remains unchanged. The vertical shift for columns in between is interpolated linearly within the range $[0, \text{shift of the first column}]$. This gradual adjustment ensures a smooth vertical transition across the panorama.